

PHP VELHO OESTE 2026

Integração de LLMs com PHP e Laravel

Construindo um Agente Real

João Salomão

Senior Software Engineer at UserGems

Uma curiosidade...

PHP Velho Oeste 2026



FUNDAMENTOS

O que é uma LLM?

Large Language Model — um modelo de linguagem treinado em bilhões de parâmetros que recebe um prompt e gera texto, token por token, de forma probabilística.

LLM — o foco desta talk

- Input: texto (prompt)
- Output: texto ou JSON estruturado
- Capacidades: raciocínio, código, análise, conversação, tool use
- Ex.: GPT-5.5, Claude Opus 4.7, Gemini 3

Outros modelos de IA

- Geração de imagens (DALL·E, Midjourney)
- Geração de áudio (ElevenLabs, Suno)
- Geração de vídeo (Sora, Runway)
- Speech-to-text (Whisper)

não são o foco aqui

PROVIDERS

Principais providers

OpenAI

GPT-5.5, GPT-5.4 mini,
GPT-5 nano

Anthropic

Claude Opus, Sonnet,
Haiku

Google

Gemini 3.1 Pro / Flash /
Flash-lite

Meta / Llama

Llama 4 Scout / Maverick

xAI / Grok

Grok 4.3

Mistral AI

Large, Medium, Small

DeepSeek

V4, V3, R1

Maritaca.ai

Sabiá-4, Sabiazinho 4 (BR)

PROTOCOLO

Comunicação com a LLM

Tudo começa com uma requisição HTTP POST.

●●● POST /v1/messages

```
curl https://api.anthropic.com/v1/messages \  
--header "x-api-key: YOUR_API_KEY" \  
--header "Content-Type: application/json" \  
--data '{  
  "model": "claude-sonnet-4-6",  
  "max_tokens": 1024,  
  "messages": [  
    {"role": "user", "content": "Hello, Claude"}  
  ]  
'
```

Todas as APIs de LLM seguem esse padrão: POST com JSON, modelo, mensagens e parâmetros.

PROTOCOLO

Streaming com SSE

Adicione "stream": true e receba tokens em tempo real via Server-Sent Events.

Anthropic

```
event: message_start  
data: {"type":"message_start"...}
```

```
event: content_block_delta  
data: {"delta":{"text":"The"}}
```

```
event: content_block_delta  
data: {"delta":{"text":" sea"}}
```

```
event: content_block_delta  
data: {"delta":{"text":" is vast."}}
```

```
event: content_block_stop  
event: message_stop
```

OpenAI

```
event: response.created  
data: {"type":"response.created"...}
```

```
event: response.output_text.delta  
data: {"delta":"Hello"}
```

```
event: response.output_text.delta  
data: {"delta":"!"}
```

```
event: response.output_text.delta  
data: {"delta":" How can I..."}
```

```
event: response.output_text.done  
event: response.completed
```

Mesmo conceito, nomes de eventos diferentes

OUTPUT

Tipos de output

O modelo pode retornar texto livre ou dados estruturados.

Text output



"Quantum computing uses qubits
that can exist in multiple states
at once, enabling parallel
computation..."

Chatbots, resumos, análises, redação

Structured output



input → schema

```
{ "prompt": "Analyze sentiment...",  
  "schema": {  
    "sentiment": enum[pos|neg|neu],  
    "score": number,  
    "topics": string[]  
  } }
```



output

```
{  
  "sentiment": "positive",  
  "score": 0.92,  
  "topics": ["ai", "php"]  
}
```

Extração de dados, classificação, pipelines

TOOLS

Tool calling

O modelo decide quais funções chamar com base no contexto.

1. Tool definition

enviada para a API

```
{  
  "tools": [{  
    "name": "lookup_order",  
    "description": "Looks up an order's status",  
    "input_schema": { "type": "object",  
      "properties": { "id": { "type": "string" } }  
    }  
  ]  
}
```

2. User prompt

enviada para a API



What's the status of
order ORD-1234?"

3. LLM response

stop_reason = tool_use

```
{  
  "stop_reason": "tool_use",  
  "content": [{  
    "type": "tool_use",  
    "name": "lookup_order",  
    "input": { "id": "ORD-1234" }  
  }]  
}
```

4. Tool runs, LLM finalizes

Texto natural de resposta



Tool response: { "status": "shipped" }

LLM final response: Order ORD-1234 was shipped
yesterday and arrives Friday."

O que é um agente de IA?

Uma LLM em loop, com tools, decidindo sozinho até resolver uma tarefa.

Anatomia

- LLM raciocina
- Tools = funções que ele pode chamar
- Loop: pensa → chama → observa → repete
- Stop condition (end_turn, max steps, timeout)

Chatbot vs. Agente

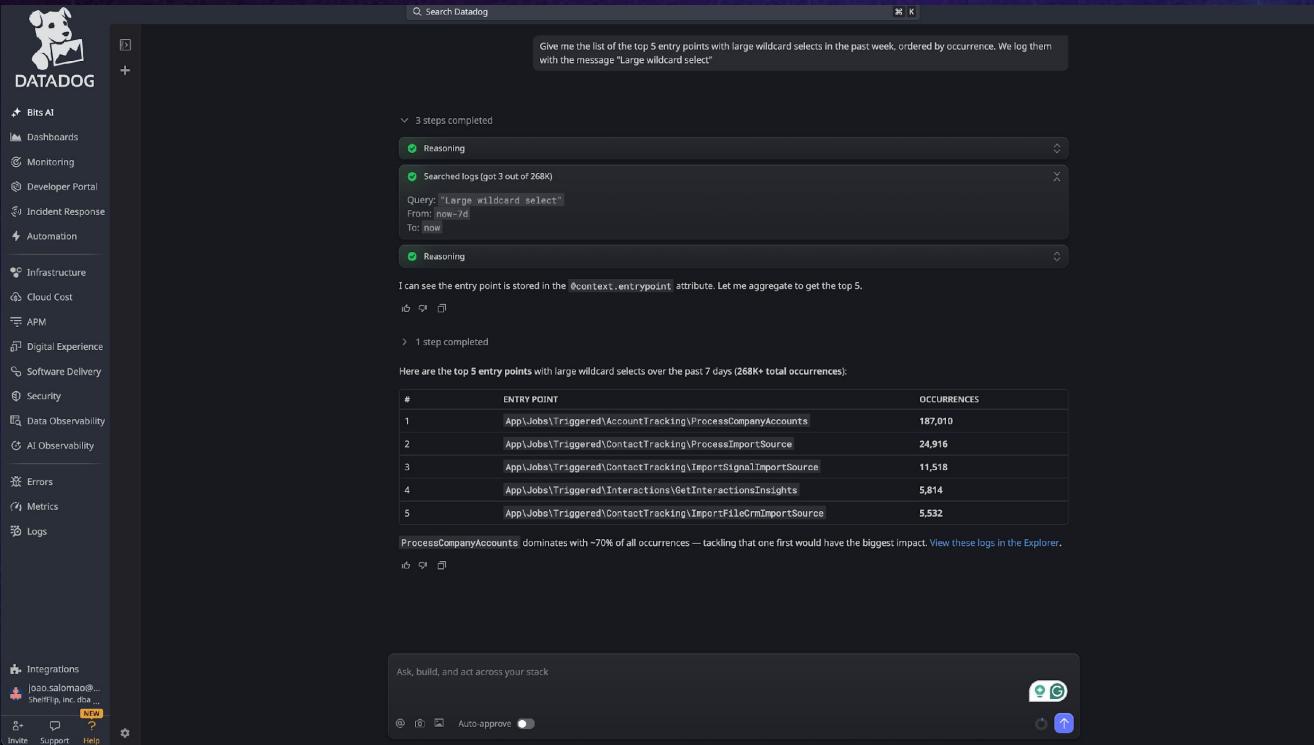
Chatbot	Agente
1 req → 1 resp	N iterações até resolver
Passivo	Decide sozinho o que chamar
Sem efeito colateral	Executa ações no mundo

Vamos ver isso na prática...

AGENTES

Exemplo real: Bits AI (Datadog)

Um agente de observabilidade analisando logs de uma app PHP/Laravel.



Search Datadog

Give me the list of the top 5 entry points with large wildcard selects in the past week, ordered by occurrence. We log them with the message "Large wildcard select"

3 steps completed

- Reasoning
- Searched logs (got 3 out of 268K)
Query: "Large wildcard select"
From: now-7d
To: now
- Reasoning

I can see the entry point is stored in the `@context.entrypoint` attribute. Let me aggregate to get the top 5.

1 step completed

Here are the top 5 entry points with large wildcard selects over the past 7 days (268K+ total occurrences):

#	ENTRY POINT	OCCURRENCES
1	<code>App\Jobs\Triggered\AccountTracking\ProcessCompanyAccounts</code>	187,010
2	<code>App\Jobs\Triggered>ContactTracking\ProcessImportSource</code>	24,916
3	<code>App\Jobs\Triggered>ContactTracking\ImportSignalImportSource</code>	11,518
4	<code>App\Jobs\Triggered\Interactions\GetInteractionsInsights</code>	5,814
5	<code>App\Jobs\Triggered>ContactTracking\ImportFileCrmImportSource</code>	5,532

`ProcessCompanyAccounts` dominates with ~70% of all occurrences — tackling that one first would have the biggest impact. View these logs in the Explorer.

Ask, build, and act across your stack

Auto approve

1 Reasoning

entende a tarefa

2 Searched logs

"large wildcard select", last 7d

3 Reasoning

decide agregar por `@context.entrypoint`

4 Resposta

top-5 + insight ~70% do total

Pergunta: "Give me the top 5 entry points with large wildcard selects in the past week, ordered by occurrence."

ECOSSISTEMA

SDKs e o problema do multi-provider

Cada provider tem seu próprio SDK. E se ele cair?

O problema

Provider fora do ar, rate limit, troca de modelo — seu código quebra.

A solução

SDK multi-provider com failover automático e interface unificada.

Opções no ecossistema PHP

PrismPHP

Community, Laravel

Symfony AI

Oficial Symfony

NeuronAI

Community, agent-first

Laravel AI SDK

First-party Laravel

PrismPHP

Interface unificada para LLMs no Laravel · community-driven

Multi-provider · Tool calling · Structured output · Streaming · **MCP client (via Relay)**

1. Definir a tool

```
● ● ●  
$orderTool = Tool::as('lookup_order')  
->for("Looks up an order's status")  
->withStringParameter(  
    'id', 'Order ID'  
)  
->using(  
    fn (string $id): string =>  
        Order::find($id)->toJson()  
);
```

2. Usar com a LLM

```
● ● ●  
$response = Prism::text()  
->using(  
    Provider::Anthropic,  
    'claude-sonnet'  
)  
->withMaxSteps(5)  
->withTools([$orderTool])  
->withPrompt('Status of order ORD-1234?')  
->asText();
```

Symfony AI & NeuronAI

Symfony AI

Oficial Symfony

70+ pacotes, parceria com PHP Foundation pro MCP SDK oficial. Multi-agent orchestration com handoffs.

- Platform (25+ providers)
- Agent (tools, workflows)
- Store (vector DBs / RAG)
- Chat (histórico)
- Mate (MCP server)
- MCP Bundle (SDK oficial)

NeuronAI

Community · agent-first · Laravel + Symfony + vanilla PHP

Framework focado em agentes/workflows. Você define Agents e Tools como classes PHP. RAG nativo, observabilidade embutida (Inspector).

- Agents como classes (extends Agent)
- Tools como classes (extends Tool)
- RAG (Pinecone, Qdrant, pgvector)
- Workflows / multi-agent
- MCP support
- Observability built-in

Laravel AI SDK

First-party · estável com Laravel 12 · API unificada para múltiplos providers.

Features

- `make:agent` / `make:tool` — scaffolding
- PHP attributes: `#[Provider]`, `#[Model]`, `#[MaxSteps]`, `#[Temperature]`
- Streaming nativo via `→stream()` (SSE)
- RAG built-in: `vector()` column + `SimilaritySearch`
- Embeddings: `Str::of(...)→toEmbeddings()`
- Failover entre providers

```
App\Ai\Agents\SalesCoach

use Laravel\Ai\Attributes\Model, Provider;
use Laravel\Ai\Concerns\RemembersConversations;
use Laravel\Ai\Contracts\Agent, Conversational;
use Laravel\Ai\Enums\Lab;
use Laravel\Ai\Promptable;

#[Provider(Lab::Anthropic)]
#[Model('claude-sonnet-4-6')]
class SalesCoach implements Agent, Conversational {
    use Promptable, RemembersConversations;

    public function instructions(): string {
        return 'You are a sales coach.';
    }
}
```


DEMO

Construindo o agente de suporte

Web chat que ajuda atendentes humanos a triagem e resolução de tickets — streaming + tool inspector ao vivo.

list_customers

customers + ticket count (filtro opcional)

search_tickets

filtro por status, keyword, customer_id

update_ticket_status

muda status (com confirmação humana)

Stack

Modelo	Claude Sonnet 4.6 (Anthropic)
SDK	Laravel/Ai
UI	Inertia + Vue 3 + Tailwind
Transport	Server Sent Events (SSE)
DB	SQLite (tickets, customers)
MCP	Laravel/MCP

Arquitetura

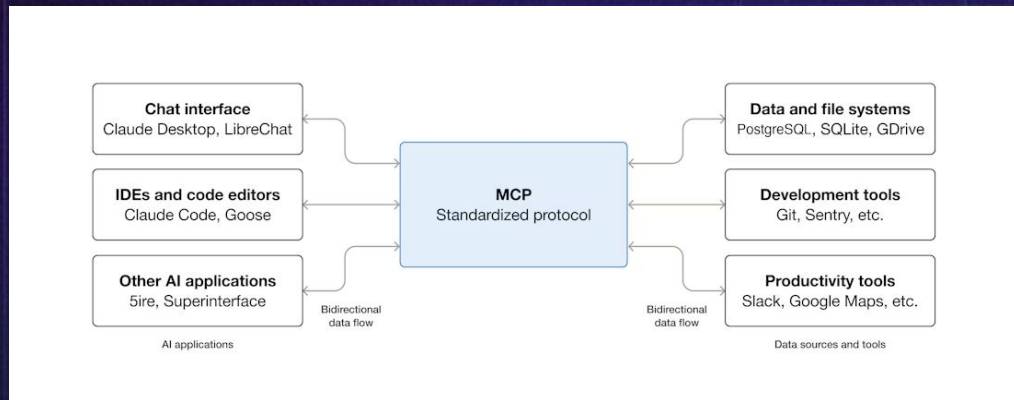
```
Browser (Vue + Inertia)
  | POST /chat/stream
  ▼
ChatController::stream()
  ↓ SupportAgent::make(...)
  ↓ ->stream()
  ▼
fetch + ReadableStream → chat
(bubbles + tool inspector)
```

Demo chat app

MCP

E o protocolo MCP?

Model Context Protocol — padrão aberto para conectar agentes a tools, dados e contextos externos. Mantido por Anthropic + Microsoft + PHP Foundation. Stdio e HTTP como transports. Sem vendor lock-in



Os servidores MCP podem fornecer três tipos principais de funcionalidades:

Resources: Dados semelhantes a arquivos que podem ser lidos pelos clientes (como respostas de API ou conteúdo de arquivos)

Tools: Funções que podem ser chamadas pelo LLM (com aprovação do usuário)

Prompts: Instruções predefinidas que ajudam os usuários a realizar tarefas específicas

Uma tool escrita uma vez vira capacidade de qualquer cliente MCP — Claude Desktop, Cursor, ChatGPT, agente do colega ao lado.

Demo MCP server

Recomendações e lições aprendidas

- Modelo barato exige prompts mais detalhados e mais exemplos
- CAPS ALTA enfatiza instruções críticas (e.g Claude Code rage detector)
- Upgrade de modelos = upgrade de dependências — não deixe pra depois (e.g Claude Sonnet 4 → Claude Sonnet 4.6)
- Monitore custos e qualidade desde o dia 1 (LangFuse, Datadog, application alarms, limites por cliente, etc.)
- Batch API = 50% de desconto se não precisa de resposta em real-time (prism-php/prism PR #951)
- Modelo certo pro job certo — teste antes de casar com um provider (e.g Opus vs GPT for writing)
- Leia a doc do provider. Cada um pode ter características que impactam seu use-case
- Protocolos: Agent-to-Agent, Agent-to-UI, Agent User Interaction Protocol, MCP
- No/Low code agents builder apps: OpenAI Agent Builder, n8n, LangSmith's, etc.

FECHAMENTO

O ecossistema PHP está pronto para AI.

prismphp.comai.symfony.comlaravel.com/aineuron-ai.dev

Obrigado! · João Salomão

linkedin.com/in/joao-salomao-js/github.com/joao-salomao/talk-php-velho-oeste-2026